

SATA Synthetic Experiment Pipeline

Implementation Specification for Bug-Hunting

SATA Project — Honours Thesis

September 2026

Contents

I	Data Generation	3
1	Overview	3
1.1	End-to-End Pipeline	3
2	Configuration Parameters (NB01)	3
3	Feature Roles	3
4	Task Sampling: <code>_sample_tasks()</code>	4
5	Rule Families: <code>_apply_rule()</code>	4
5.1	Linear	4
5.2	Threshold (Majority Vote)	4
5.3	Tree (Depth-3)	4
5.3.1	Tree Leaf Label Sampling	5
5.4	Sparse Interaction	5
6	Environment Generation: <code>generate_environment()</code>	5
6.1	<code>id</code> — In-Distribution	5
6.2	<code>covariate</code> — $P(X)$ Shift	5
6.3	<code>mechanism</code> — $P(Y X)$ Shift	5
6.4	<code>spurious_reversal</code> — Spurious Flip	6
6.5	<code>extrapolation</code> — Out-of-Support	6
6.6	<code>missing_feature</code> — Broken Measurement	6
7	Spurious Feature Construction (f_8)	6
8	Label Noise	6
9	Dataset Construction: <code>task_to_dataframes()</code>	6
10	Causal Graph	7
II	Downstream Pipeline	8

11 Demo Selection (NB02)	8
11.1 Zero-Shot	8
11.2 Random	8
11.3 Similarity	8
11.4 Label Diversity (Protocol A)	8
11.5 Feature Range (Protocol B)	8
11.6 Rule Diversity (Protocol C)	8
11.7 Counter-Spurious (Protocol D)	8
11.8 Ordering (Applied to All Methods)	9
12 Serialisation	9
13 Prompt Construction	9
14 LLM Inference	9
14.1 Forward Pass	9
14.2 Prediction Extraction	10
15 Evaluation Metrics	10
15.1 Accuracy	10
15.2 Macro F1	10
15.3 Shift Gap	10
15.4 Invalid Rate	10
16 Experiment Orchestration (NB02)	11
III Bug-Hunting Notes	12

Part I

Data Generation

1 Overview

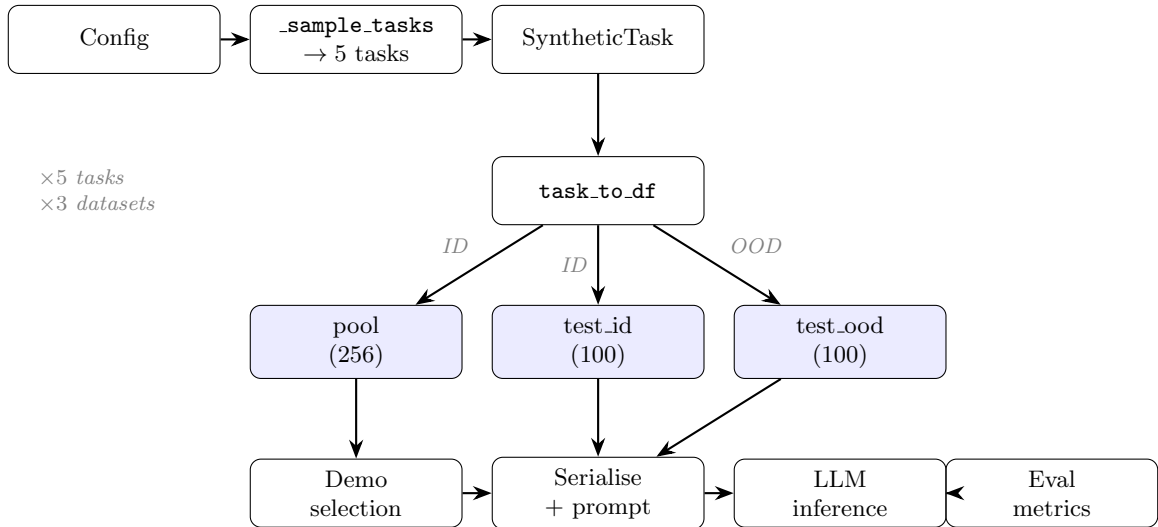
The generator produces synthetic binary-classification tasks with *known ground truth*, so that we can precisely control which distribution shift is applied and measure whether an LLM’s in-context learning (ICL) is affected by different demo-selection strategies.

Two independent axes define every dataset:

1. **Rule family** — how labels are generated from features (linear, threshold, tree, sparse_interaction).
2. **Environment / shift type** — how the OOD test set differs from the ID data (covariate, mechanism, spurious_reversal, extrapolation, missing_feature).

The same set of tasks is reused across all three primary datasets; only the OOD environment changes.

1.1 End-to-End Pipeline

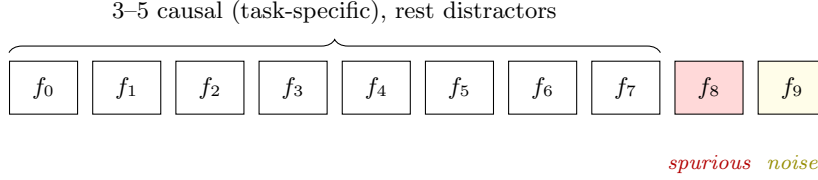


2 Configuration Parameters (NB01)

Parameter	Value	Meaning
n_features	10	Total columns: $f_0, \dots, f_9$
n_causal_range	[3, 5]	$n_{\text{causal}} \sim \text{U}\{3, 4, 5\}$
rule_families	linear, threshold, tree	Sampled uniformly
spurious_strength_range	[0.80, 0.90]	$s \sim \text{U}(0.80, 0.90)$
label_noise	0.02	$P(\text{flip}) = 0.02$
coefficient_scale	1.5	$c_j \sim \mathcal{N}(0, 1.5^2)$
N_TASKS	5	Tasks per experiment
N_POOL	256	Demo candidate pool per task
N_TEST_ID	100	ID test rows per task
N_TEST_OOD	100	OOD test rows per task

3 Feature Roles

Every row has 10 features:



- f_0 – f_7 : drawn $\sim \mathcal{N}(0, 1)$. A task-specific subset (3–5) is *causal*; the rest are distractors.
- f_8 : **spurious**, generated *from* the label (Section 7). Not used by any rule.
- f_9 : **pure noise**, $\sim \mathcal{N}(0, 1)$, independent of everything.

4 Task Sampling: `_sample_tasks()`

For each of the $N = 5$ tasks, sampled independently:

1. **Rule family:** $\text{family} \sim \text{Uniform}\{\text{linear}, \text{threshold}, \text{tree}\}$
2. **Number of causal features:** $n_{\text{causal}} \sim \text{U}\{3, 4, 5\}$
3. **Causal feature indices:** n_{causal} distinct indices from $\{0, \dots, 7\}$, sorted.
4. **Coefficients** (used only by linear, sampled for all families for RNG consistency): $c_j \sim \mathcal{N}(0, 1.5^2)$
5. **Spurious strength:** $s \sim \text{U}(0.80, 0.90)$
6. **Family-specific:**
 - **threshold:** $k = \min(3, n_{\text{causal}})$ thresholds, $t_j \sim \text{U}(-0.3, 0.3)$.
 - **tree:** $k = \min(3, n_{\text{causal}})$ leaf labels via `_sample_tree_leaf_labels(k)` (Section 5.3.1).
 - **sparse_interaction:** threshold from a 10k-sample probe targeting base rate $\sim \text{U}(0.35, 0.65)$.

5 Rule Families: `_apply_rule()`

Given $X \in \mathbb{R}^{n \times 10}$ and optional $u \in [0, 1]^n$, compute $y_{\text{clean}} \in \{0, 1\}^n$:

5.1 Linear

$$\text{logit}_i = \mathbf{c}^\top \mathbf{X}_{i, \text{causal}}, \quad p_i = \text{sigmoid}(\text{logit}_i) = \frac{1}{1 + e^{-\text{logit}_i}}$$

$$y_{\text{clean}, i} = \begin{cases} \mathbf{1}[u_i < p_i] & \text{stochastic (data generation)} \\ \mathbf{1}[p_i > 0.5] & \text{MAP (probes)} \end{cases}$$

Why stochastic? A deterministic rule is scale-invariant, making the causal signal perfectly reliable at any coefficient scale. Bernoulli sampling introduces concept noise so the spurious feature can compete.

5.2 Threshold (Majority Vote)

$k = \min(3, n_{\text{causal}})$, thresholds t_1, t_2, t_3 :

$$b_{i,j} = \mathbf{1}[X_{i, \text{causal}_j} > t_j], \quad y_{\text{clean}, i} = \mathbf{1}\left[\sum_{j=1}^k b_{i,j} \geq 2\right]$$

5.3 Tree (Depth-3)

$k = \min(3, n_{\text{causal}})$, giving 2^k leaves:

$$\text{leaf_id}_i = \sum_{j=0}^{k-1} \mathbf{1}[X_{i, \text{causal}_j} > 0] \cdot 2^j, \quad y_{\text{clean}, i} = \text{leaf_labels}[\text{leaf_id}_i]$$

5.3.1 Tree Leaf Label Sampling

Two constraints:

1. **Complement antisymmetry:** $\text{leaf_labels}[L] = 1 - \text{leaf_labels}[2^k - 1 - L]$. Guarantees mechanism's sign flip always inverts the label.
2. **Non-degeneracy** ($k \geq 3$): Reject dictator (single-bit) and parity (XOR-of-all-bits) functions.

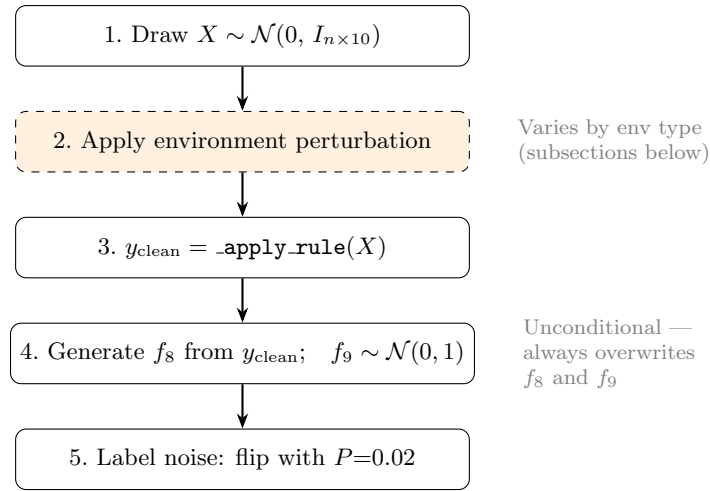
5.4 Sparse Interaction

$$y_{\text{clean}} = \mathbf{1}[X_{:,i} \cdot X_{:,j} > \theta]$$

Only the first two causal features participate; the rest are decoys.

6 Environment Generation: `generate_environment()`

Every call follows the same skeleton:



6.1 id — In-Distribution

No perturbation. X , $\mathbf{c}$, s unchanged.

6.2 covariate — $P(X)$ Shift

Shift $P(X)$ while preserving $P(Y)$:

1. Estimate unshifted base rate $r_{\text{id}} = \hat{P}(y=1)$ via 2048-row probe.
2. **Rejection loop** (up to 50 iterations):
 - (a) Sample 3 features from $\{0, \dots, 7\}$ without replacement.
 - (b) Sample deltas: $\delta_j \sim \text{U}(1, 2) \times \text{Rademacher}$ (so $\delta_j \in [-2, -1] \cup [1, 2]$).
 - (c) Probe shifted base rate r_{shifted} .
 - (d) Accept if $|r_{\text{shifted}} - r_{\text{id}}| \leq 0.03$.
3. Apply: $X_{:, \text{feats}} += \delta$.
Features 8, 9 excluded (overwritten later).

6.3 mechanism — $P(Y|X)$ Shift

1. Rescale coefficients: $c'_j = c_j \cdot u_j$, $u_j \sim \text{U}(0.5, 1.5)$
2. Sign-flip causal features (on a copy): $\tilde{X}_{:, \text{causal}} = -X_{:, \text{causal}}$

The rule operates on $\tilde{X}$ with c' ; the returned X is unchanged. The LLM sees original features but shifted labels.

Note on f_8 : Generated from y_{clean} *after* the mechanism flip, so f_8 tracks the *new* ground truth at strength s .

6.4 spurious_reversal — Spurious Flip

One line: $s' = 1 - s$. Everything else unchanged. $P(\text{agreement})$ drops from ~ 0.85 to ~ 0.15 .

6.5 extrapolation — Out-of-Support

Replace 2–3 causal features per row: $X_{i,f} = s_i \cdot m_i$, $m_i \sim U(2, 4)$, $s_i \sim \text{Rademacher}$.

6.6 missing_feature — Broken Measurement

$X_{:, \text{causal}[0]} \leftarrow \mathcal{N}(0, 1)$ (fresh, independent of label).

7 Spurious Feature Construction (f_8)

$$d = \Phi^{-1}(s), \quad X_{i,8} = (2y_{\text{clean},i} - 1) \cdot d + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 1)$$

Distribution by label:

$$y = 1 \implies X_8 \sim \mathcal{N}(+d, 1)$$

$$y = 0 \implies X_8 \sim \mathcal{N}(-d, 1)$$

Sign-agreement: For $y = 1$: $P(X_8 > 0) = P(Z > -d) = \Phi(d) = s$. For $y = 0$: $P(X_8 < 0) = P(Z < d) = \Phi(d) = s$.

Under spurious reversal: $d' = \Phi^{-1}(1 - s) = -d$, so $P(\text{agreement}) = 1 - s$.

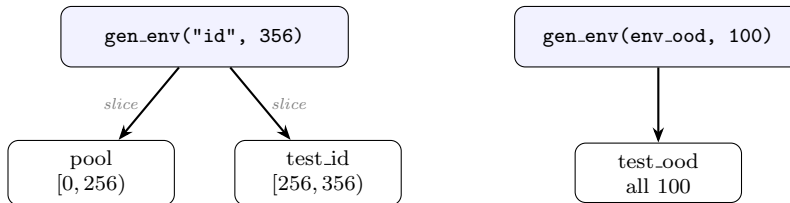
8 Label Noise

$$y_i = \begin{cases} 1 - y_{\text{clean},i} & \text{w.p. } \eta = 0.02 \\ y_{\text{clean},i} & \text{w.p. } 1 - \eta \end{cases}$$

f_8 is generated from y_{clean} (pre-noise), not y (post-noise).

9 Dataset Construction: task_to_dataframes()

For one task and one OOD environment:

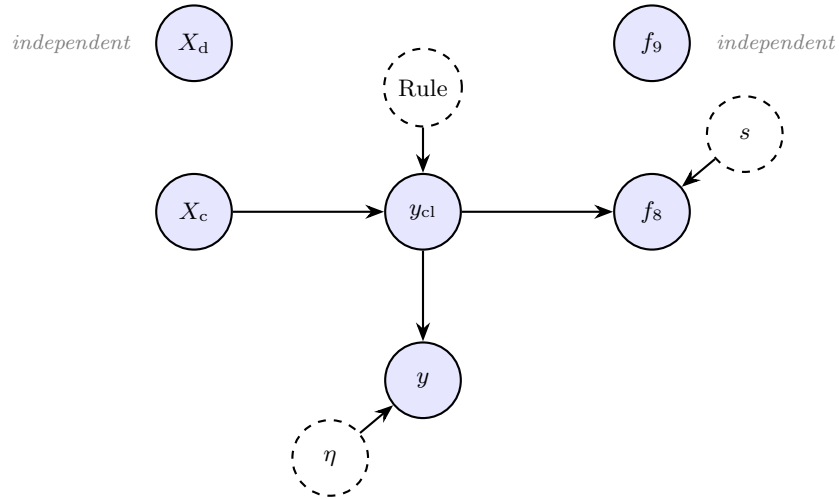


Repeated for $5 \text{ tasks} \times 3 \text{ datasets}$. Each dataset uses a different OOD:

Dataset	env_ood	What shifts
synth_covariate	covariate	$P(X)$
synth_concept	mechanism	$P(Y X)$
synth_spurious	spurious_reversal	f_8 correlation

Sizes per dataset: $\text{pool} = 5 \times 256 = 1280$, $\text{test_id} = 500$, $\text{test_ood} = 500$.

10 Causal Graph



$$X_{\text{causal}} \xrightarrow{\text{rule}} y_{\text{clean}} \xrightarrow{(2y-1)d+\varepsilon} f_8$$

f_8 is a *descendant* of y , not a cause.

Part II

Downstream Pipeline

11 Demo Selection (NB02)

All selection functions return `list[int]` — indices into the pool DataFrame. $K = 8$ demos per query.

11.1 Zero-Shot

No demos selected. The LLM receives only the system instruction and query.

11.2 Random

K indices sampled uniformly without replacement from the pool:

$$\text{demos} \sim \text{Uniform}(\text{pool.index}, K)$$

11.3 Similarity

Each pool row and query are serialised to text, then encoded via `all-MiniLM-L6-v2` sentence embeddings (normalised). Cosine similarity is a dot product:

$$\text{sim}_j = \mathbf{e}_{\text{pool}_j}^\top \mathbf{e}_{\text{query}}, \quad \text{demos} = \text{argsort}(-\text{sim})[:K]$$

Pool embeddings are cached (keyed by hash of pool texts, max 64 entries). Batch variant computes $Q \times P$ similarity matrix in one call.

11.4 Label Diversity (Protocol A)

Stratified sampling: $\lfloor K/2 \rfloor$ demos per class. Remainder (if K is odd) filled randomly from leftover pool.

11.5 Feature Range (Protocol B)

For up to 3 features from `top_features` (ranked by mutual information):

1. Bin each feature into $n_{\text{bins}} = 3$ quantiles via `pd.qcut`.
2. Form Cartesian cell key per row: $(\text{bin}_{f_1}, \text{bin}_{f_2}, \text{bin}_{f_3})$.
3. Shuffle cells, sample 1 demo per cell until K reached.
4. Remainder filled randomly.

11.6 Rule Diversity (Protocol C)

Two arms:

- **Real arm:** Fit a `max_depth=3` decision tree on the pool; use `tree.apply()` to get leaf IDs as regime proxies.
 - **Synthetic arm:** Use ground-truth `regime` from metadata.
- Group pool rows by leaf/regime, shuffle groups, take 1 per group.

11.7 Counter-Spurious (Protocol D)

Two arms:

- **Synthetic arm:** Use ground-truth `is_counter_spurious` boolean from metadata.
- **Real arm:** Find proxy features by $\text{score}_f = |\text{corr}(f, \text{label})| \cdot |\text{corr}(f, \text{shift})|$. Identify “minority cells” where proxy–label correlation is broken.

Over-sample counter-spurious rows: $\lfloor K/2 \rfloor + 1$ from counter-spurious indices, remainder from the rest.

11.8 Ordering (Applied to All Methods)

After selection, demo order is shuffled via a seeded permutation:

$$\text{ordered_ids} = \pi_{\text{seed}}(\text{demo_ids})$$

This is a controlled nuisance variable (Lu et al., 2022): ordering effects are uniform across all conditions.

12 Serialisation

Each row is converted to text via `serialise_row()`:

```
f0: 0.80; f1: -0.45; f2: -0.30; f3: 1.52; f4: -0.91;  
f5: 1.10; f6: 0.07; f7: -1.33; f8: 0.64; f9: 0.22 -> 1
```

Format: FeatureName: value; ... -> Label

- Feature order: alphabetical by column name (`sorted(features.keys())`).
- Numeric values: fixed-width `f"{v:.2f}"` (2 decimal places).
- Query rows: end with " ->" (no label).
- NaN values: rendered as "unknown".
- When codebook is provided: column names replaced by `name_extended`, categorical codes replaced by human-readable labels.

13 Prompt Construction

`build_chat_messages()` returns a 2-element message list:

1. **System message:** "You are a classifier. Given the input features, predict the class. Respond with exactly one word: 0 or 1, where 0 = Class 0 and 1 = Class 1."
2. **User message:** Newline-joined demo lines (each with label), followed by the query line (no label).

`ChatFormatter.render(system, user)` applies the model's chat template (e.g. ChatML for Qwen) with special tokens and generation prompt:

```
<|im_start|>system  
You are a classifier. ...  
<|im_end|>  
<|im_start|>user  
f0: 0.80; f1: -0.45; ... -> 1  
f0: 1.20; f1: 0.33; ... -> 0  
...  
f0: -0.10; f1: 0.72; ... ->  
<|im_end|>  
<|im_start|>assistant
```

14 LLM Inference

Using `MLXRunner` (Apple Silicon, Qwen2.5-7B-Instruct).

14.1 Forward Pass

For each prompt, `next_token_logprobs(prompt)` performs:

1. Tokenise the full prompt string.
2. Single forward pass through the model.
3. Extract logits at the *final* token position.

4. Compute full-vocabulary log-probabilities:

$$\ell_v = \text{logits}_v - \text{logsumexp}(\text{logits})$$

No sampling occurs; the model produces a probability distribution over all tokens and we extract the two we care about.

14.2 Prediction Extraction

`get_confidence(logprobs_dict, label_tokens)` computes:

1. Look up log-probs for the two label tokens:

$$\ell_0 = \ell[\text{token}("0")], \quad \ell_1 = \ell[\text{token}("1")]$$

If a label token is absent from logprobs, default $\ell = -100$.

2. **Constrained two-way softmax** (renormalise over label tokens only):

$$p_0 = \frac{e^{\ell_0}}{e^{\ell_0} + e^{\ell_1}}, \quad p_1 = 1 - p_0$$

3. Prediction: $\hat{y} = \arg \max(p_0, p_1)$.

Confidence: $\max(p_0, p_1)$.

Output: `PredictionResult(prediction, confidence, p0, p1, logprob_0, logprob_1)`.

If neither label token appears in logprobs $\rightarrow$ `INVALID`.

15 Evaluation Metrics

All metrics filter out `INVALID` predictions first. Predictions are canonicalised: strip whitespace, convert numeric-looking strings to integers ("`1.0`" $\rightarrow$ "`1`").

15.1 Accuracy

$$\text{acc} = \frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbf{1}[\hat{y}_i = y_i]$$

where $\mathcal{V}$ is the set of non-`INVALID` predictions. Returns NaN if $|\mathcal{V}| = 0$.

15.2 Macro F1

$$\text{F1}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c = \frac{1}{C} \sum_{c=1}^C \frac{2 \cdot \text{precision}_c \cdot \text{recall}_c}{\text{precision}_c + \text{recall}_c}$$

Via `sklearn.metrics.f1_score(..., average="macro")` on canonicalised valid predictions.

15.3 Shift Gap

$$\Delta = \text{acc}_{\text{ID}} - \text{acc}_{\text{OOD}}$$

Positive Δ indicates OOD degradation.

15.4 Invalid Rate

$$\text{inv} = \frac{|\{i : \hat{y}_i = \text{INVALID}\}|}{n}$$

16 Experiment Orchestration (NB02)

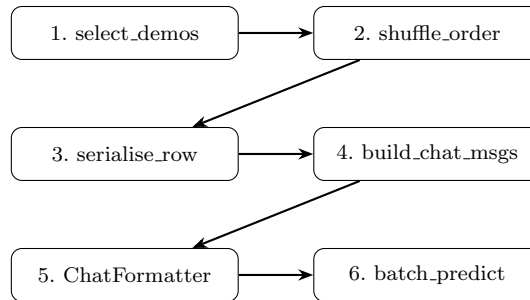
Parameters: $K = 8$ demos, $N_{\text{queries}} = 30$ per environment, seed = 42.

Conditions: zero_shot, random, similarity, label_diversity, feature_range, rule_diversity, counter_spurious (7 total).

Per-dataset pre-computation:

1. Load parquet files (pool, test_id, test_ood).
2. `select_top_features(pool, n=3)` for feature_range.
3. `fit_leaf_tree(pool)` for rule_diversity.
4. `find_spurious_proxy_features(pool  $\cup$  test_ood)` for counter_spurious.
5. Pre-compute similarity embeddings (batch).

Per-query data flow:



All prompts for a condition are batched, then `runner.batch_predict(prompts, ("0", "1"))` runs them through MLX. Results are scored per environment with accuracy, macro_f1, invalid_rate.

Part III

Bug-Hunting Notes

1. **f_8 is present in ALL environments (including ID).** The spurious correlation is part of the data-generating process. Only `synth_spurious` tests reliance on f_8 by reversing it.
2. **f_8 is a “safety net” in covariate and concept shifts.** In these datasets, f_8 ’s strength is unchanged between ID and OOD. In `synth_concept`, f_8 tracks the *new* ground truth (generated after the mechanism flip), so it can help the LLM appear to generalise when it is actually following a spurious cue.
3. **Covariate shift only touches f_0 – f_7 .** f_8 and f_9 are overwritten unconditionally.
4. **Mechanism shift: LLM sees original X , not $\tilde{X}$.** The sign flip is on the copy fed to the rule.
5. **Probes produce throwaway data.** Monte Carlo estimates for the rejection loop are not part of the final dataset.
6. **Concept noise u drawn for all families.** Ensures RNG stream is family-independent for reproducibility.
7. **Sparse interaction’s extra “causal” features are decoys.** Only the first two indices participate in the product.
8. **Label noise is after f_8 generation.** f_8 tracks y_{clean} , not y_{final} . $\sim 2\%$ of rows have f_8 misaligned with the noised label.
9. **Constrained two-way softmax discards all non-label tokens.** If the model is highly uncertain between “0” and “1” but confident about some other token (e.g. “The”), the renormalised p_0, p_1 can still be near 0.5 — this is by design (we only care about the relative preference for the two valid labels).
10. **Feature ordering is alphabetical.** `sorted(features.keys())` on names like `f0`, `f1`, ..., `f9` gives $f_0, f_1, \dots, f_9$ — correct because all names are equal-length single-digit. If features were `f0`, ..., `f10`, alphabetical sort would give `f0`, `f1`, `f10`, `f2`, ... — a potential serialisation bug for > 10 features.
11. **Demo ordering is a controlled nuisance.** `shuffle_order()` applies the same seeded permutation across all conditions, so ordering effects do not confound condition comparisons.
12. **Similarity selection is on raw text, not features.** The embedding model sees the serialised string including feature names, colons, and semicolons. Similarity is computed in embedding space, not feature space. Whether this is a feature or a bug depends on whether the serialisation format adds meaningful signal.